

First-Class Constructor Subsets for Pattern Matching on Indexed Families

Anonymous author

Anonymous affiliation

Abstract

Pattern matching is a powerful feature of functional and dependently typed languages, yet programmers often encounter unreachable clauses. This either leads to unsafe partial functions or, in safe settings, to proof obligations that require tedious resolution of trivial cases. The problem is particularly acute when programs operate on subsets of an inductive datatype. Standard encodings pair data with propositions or refactor datatypes into indexed families, but both approaches complicate composition and proof automation.

We present a subtyping discipline for constructor subsets, where first-class signatures describe which constructors of a family are available. These signatures make the intended subset explicit at the type level, enabling pattern matching to rule out impossible branches without generating spurious obligations. Our prototype indicates that this approach can eliminate many trivial proof obligations while preserving type safety.

We implemented a small proof assistant that supports constructor subsets (signatures), and we developed a small prototype that translates its theory to the `Lambdapi` framework. We can encode the entire type system in fewer than 600 lines, supporting our claim that the approach is simple and lightweight.

2012 ACM Subject Classification Theory of computation $\rightarrow$ Program verification; Theory of computation $\rightarrow$ Type theory

Keywords and phrases Datatype, Type Constructors, Constructor Subtyping

Digital Object Identifier 10.4230/LIPIcs...

1 Introduction

Subtyping is a fundamental concept across programming languages that enables the specialization of type definitions through hierarchical relationships between supertypes and subtypes. At a high level, a subtype describes a “smaller” collection of values than its supertype. While term-level subtyping is extensively implemented in many languages, subtyping between constructors of inductive types remains relatively unexplored, especially in the presence of dependent types.

A recurring difficulty arises when an inductive datatype is only partially available in a given context. For example, a function may be intended to consume lists that are known to be non-empty, or vectors whose length is positive. In most dependently typed languages the programmer faces a choice: either write an unsafe partial function, or thread propositions through the program and discharge proof obligations stating that certain constructors are impossible. The latter approach leads to unreachable clauses in pattern matches and to proofs that are often routine but cumbersome.

One common technique is to maintain the original datatype together with a proposition that rules out some constructors. Another approach, illustrated in Figures 1 and 2, uses indexed datatypes to restrict which constructors may appear. Both techniques have well-known drawbacks. The first ties ordinary programs to logical predicates, complicating function composition and proof reuse. The second introduces additional indices and can interact badly with proof irrelevance and unification; it may also hinder the detection of unreachable branches when constructor information flows through computations.



© Anonymous author(s);

licensed under Creative Commons License CC-BY 4.0

Leibniz International Proceedings in Informatics

LIPICs Schloss Dagstuhl – Leibniz-Zentrum für Informatik, Dagstuhl Publishing, Germany

Constructor subtyping has been explored in other settings. In O’Haskell [10], records and algebraic datatypes can be equipped with an optional subtyping relation inferred from the usage of constructors and fields. For example, a non-empty list type can be extended with an empty constructor to obtain a standard list type. More recent work on zero-cost constructor subtyping in Cedille [7, 6], on order-sorted inductive types and overloading [3, 2], and on algebraic subtyping for extensible records [8] shows how rich subtyping disciplines can be added to strongly typed systems. However, these systems are tailored to specific core calculi or to non-dependent settings, and do not directly address the combination of constructor subtyping with indexed families in a general dependently typed setting.

In this paper we focus on a simple but expressive fragment of constructor subtyping aimed at eliminating trivial proof obligations. Rather than allowing arbitrary constructor overloading, we work with *constructor subsets*: first-class types of the form $\{T :: \Phi\}$, where T is an inductive family and Φ is a finite list of its constructors. Intuitively, $\{T :: \Phi\}$ denotes those values of T whose *outermost (head) constructor*, when inspected in canonical form, is drawn from Φ . This is the information that coverage checking for pattern matching consumes directly, and it is exactly what makes types such as “non-empty lists” inhabited (the tail may still be empty).

Our contributions are as follows.

- We define a subtyping discipline for constructor subsets as a conservative extension of a $\lambda\Pi$ -style dependent type theory with inductive families. The system includes typing and subtyping rules for constructor signatures and a dedicated elimination rule for pattern matching on signature types.
- We show, through a series of examples, that constructor subsets capture common programming idioms such as non-empty lists and simple invariants on inductive families, while avoiding the proof obligations that arise when these invariants are expressed with explicit propositions.
- We describe a two-layer prototype: a `Lambdapi` backend encoding of constructor signatures and a Haskell transpiler that translates a lightweight surface fragment into signature-aware `LambdaPi` terms. This architecture keeps programs concise while preserving a direct correspondence with Rules 1–7.

Structure of the paper.

Section 2 introduces the core type system and subtyping rules. Section 3 presents case studies illustrating how constructor subsets simplify programming with indexed families. Section 4 sketches the implementation and discusses practical considerations, and Section 5 concludes with directions for future work.

Many type systems are inspired by constructor subtyping to address constructor overloading, allowing the same constructor name to be shared by different datatypes, even in pattern matching. However, its naive use can lead to problems with subject reduction, as observed by Frade [5]. Subject reduction is the property that during the normalization of a term its type is preserved; overloading constructors without care may violate this property. For the sake of simplicity, in this work we do not consider overloading between distinct datatypes. Instead we restrict subtyping to relations induced by subsets of constructors of a single inductive family. We believe this fragment is already expressive enough to avoid the trivial proof obligations discussed above, while keeping the meta-theory and implementation comparatively simple.

```

1 Inductive list (A : Set) : Set :=
2   | cons : A → list A → list A
3   | empty : list A.
4
5 Definition head {A} (x : list A) :
6   x <> empty A → A :=
7   match x return x <> empty A → A with
8   | cons _ h l ⇒ fun _ ⇒ h
9   | empty _ ⇒
10    fun f ⇒
11      match (f eq_refl) with end
12 end.

```

■ **Figure 1** Constructor exclusion in Coq with a proof obligation.

```

1 Inductive list' (A : Set) : bool → Set :=
2   | cons' : forall x, A →
3     list' A x → list' A true
4   | empty' : list' A false.
5
6 Definition head' {A} (x : list' A true) : A :=
7   match x with
8   | cons' _ _ h l ⇒ h
9   end.

```

■ **Figure 2** Constructor exclusion in Coq using an indexed datatype.

2 Type Rules

In this section, we introduce a simplified type system, enhanced with first-class signature subtyping support. This exposition assumes a basic familiarity with the concepts of type theory, especially dependent type theory. We use a standard typing judgment $\Gamma \vdash t : A$ for terms and a subtyping judgment $\Gamma \vdash A \sqsubseteq B$ between types. Contexts Γ map variables to types, and $Type$ denotes the universe of small types.

The subtyping relation is inspired by the subsumption rule of Aspinall and Compagnoni [1]:

$$\frac{\Gamma \vdash t : A \quad \Gamma \vdash A \sqsubseteq B}{\Gamma \vdash t : B}$$

We now fix some notation for telescopes and constructor environments. A *telescope* Δ is a sequence of typed variables

$$\Delta = (a_1 : A_1) \dots (a_n : A_n).$$

Given such a telescope and a type family T , we write $T \Delta$ for the application $T a_1 \dots a_n$. When a telescope is used only to record the indices of a datatype we write Δ^* ; in examples we use $Vector A n$ with $\Delta = (A : Type)(n : Nat)$ and $\Delta^* = (A, n)$ [11].

To refer to individual arguments in a telescope we use subscripts: if $\Delta = (a_1 : A_1) \dots (a_n : A_n)$ then Δ_i denotes $(a_i : A_i)$ and Δ_i^* denotes the corresponding index, for $1 \leq i \leq n$. For a term T we use the side condition $T_{\beta\eta}$ to indicate that we inspect the $\beta\eta$ -normal form of T .

XX:4 First-Class Constructor Subsets for Pattern Matching on Indexed Families

We write $\mathcal{C}_{\text{all}}$ for the finite set of static constructor declarations provided by the programmer. A static definition stores a constructor name. We do not rely on any particular labeling scheme; we only need to decide constructor equality, so one may use unique names or hashes. The order of static definitions is stored in $\mathcal{C}_{\text{all}}$; in particular, mutually recursive definitions can be handled by first registering every constructor in $\mathcal{C}_{\text{all}}$ and only then checking the rules below. A *signature* Φ is a finite list of constructor names drawn from $\mathcal{C}_{\text{all}}$, and a type of the form $\{T \Delta^* :: \Phi\}$ maps each constructor in Φ to the restricted type $T \Delta^*$. Formation is intentionally liberal: signatures may include constructors that are incompatible with the current index instance Δ^* , which yields uninhabited refinements but does not introduce inconsistency. We use a side predicate **AGAINST** only for elimination-time pruning (Rule 7), not for signature formation. Because $\mathcal{C}_{\text{all}}$ is finite, the side conditions that test membership in $\mathcal{C}_{\text{all}}$ are decidable.

$$\text{RULE 1} \frac{\Gamma \vdash T \Delta^* : \text{Type} \quad \Phi = (C_1, \dots, C_n) \quad (C_1 \dots C_n) \subseteq \mathcal{C}_{\text{all}} \quad \forall i, 1 \leq i \leq n, \Gamma \vdash C_i : \Delta_i \rightarrow T \Delta_i^*}{\Gamma \vdash \{T \Delta^* :: \Phi\} : \text{Type}}$$

$$\text{RULE 2} \frac{\Gamma \vdash T \Delta^* : \text{Type}}{\Gamma \vdash \{T \Delta^* :: \Phi\} \subseteq T \Delta^*}$$

$$\text{RULE 3} \frac{\Gamma \vdash T \Delta^* : \text{Type} \quad \Gamma \vdash C : \Delta_C \rightarrow T \Delta_C'^* \quad C \in \Phi \quad \Gamma \vdash \vec{u} : \Delta_C \quad (T \Delta_C'^*[\vec{u}/\Delta_C])_{\beta\eta} = (T \Delta^*)_{\beta\eta}}{\Gamma \vdash C \vec{u} : \{T \Delta^* :: \Phi\}} \quad C \in \mathcal{C}_{\text{all}}$$

$$\text{RULE 4} \frac{(T \Delta^*)_{\beta\eta} = (T' \Delta^*)_{\beta\eta} \quad \Phi' \subseteq \Phi}{\Gamma \vdash \{T' \Delta^* :: \Phi'\} \subseteq \{T \Delta^* :: \Phi\}}$$

$$\text{RULE 6} \frac{\Gamma \vdash A' \subseteq A \quad \Gamma, x : A' \vdash B \subseteq B'}{\Gamma \vdash (x : A) \rightarrow B \subseteq (x : A') \rightarrow B'}$$

119 Rule 5

We keep the following rule as an implementation-level checking convenience (not part of the declarative calculus above). It is derivable from standard application typing plus subsumption, but it is convenient to have it as a direct check in the implementation to avoid unnecessary normalization if F is not a constructor application:

$$\text{RULE 5} \frac{\Gamma \vdash F : \Delta \rightarrow A \quad \Gamma \vdash \vec{u} : \Delta \quad \Gamma \vdash A[\vec{u}/\Delta] \subseteq \{T \Delta^* :: \Phi\}}{\Gamma \vdash F \vec{u} : \{T \Delta^* :: \Phi\}} \quad F \notin \mathcal{C}_{\text{all}}$$

A term annotated as $T_{\beta\eta}$ is considered in $\beta\eta$ -normal form. In practice we only normalize enough to inspect the outermost constructor when applying Rule 3 (and the algorithmic Rule 5 check in the implementation); there is no global requirement that all terms be strongly normalizing. The choice of how aggressively to normalize is left to the implementation and presents the usual trade-off between completeness of simplification and performance.

It is straightforward to see that $\text{cons } \Delta$ has type $\{\text{Vector } \Delta^* :: |\text{cons}|\}$. One may either infer this type and then check it against an expected type T , or attempt to check $\text{cons } \Delta$ directly against T . In our prototype we follow the former strategy, inferring first and checking

second, because it tends to reduce the number of normalization steps needed during type checking.

Operational reading.

Under the usual canonical forms lemma for inductive values, a closed value of a signature type $\{T \Delta^* :: \Phi\}$ reduces (in weak head normal form) to a constructor application $C \vec{u}$ with $C \in \Phi$ and $(T \Delta_C^*[\vec{u}/\Delta_C])_{\beta\eta} = (T \Delta^*)_{\beta\eta}$. Thus, signature types refine *which head constructors* can appear at a given program point, exactly the information required to remove unreachable branches in pattern matching.

Signatures may contain constructors that are incompatible with the current indices. For example,

$$T := \{\text{Vector } A (S \ N) :: |\text{empty}| \text{cons}\}$$

is well-formed, but `empty` cannot inhabit T . These trivial subsets are harmless: they are simply uninhabited refinements, and there is no rule that introduces $\{T \Delta^* :: \Phi\}$ from $T \Delta^*$ without constructor evidence.

To eliminate unreachable branches in pattern matching, we define a pruned constructor list: $\Phi_{\text{ok}} := \text{OK}(\Gamma, T, \Delta^*, \Phi) \equiv [C \in \Phi \mid C \in \mathcal{C}_{\text{all}} \wedge \Gamma \vdash C : \Delta_C \rightarrow T \Delta_C'^* \wedge \forall j \in \{1, \dots, |\Delta^*|\}. \text{AGAINST}((\Delta_j^*), (\Delta_{C,j}'))]$, where Δ^* denotes the index tuple of type T .

The predicate `AGAINST` is given below:

`AGAINST`(p, t) is defined by the following decision procedure:

$$\text{AGAINST}(p, t) \equiv \begin{cases} \top & \text{if } p \text{ is a variable} \\ \bigwedge_{i=1}^k \text{AGAINST}(p_i, t_i) & \text{if } p = c p_1 \dots p_k \text{ and } t = c t_1 \dots t_k \\ \perp & \text{if } p = c \vec{p'} \text{ and } t = c' \vec{t'} \text{ with } c \neq c' \\ \top & \text{otherwise.} \end{cases}$$

Rule 2 allows us to forget signature information at the same index instance: any value of type $\{T \Delta^* :: \Phi'\}$ can be viewed as a value of the underlying type $T \Delta^*$. In this sense constructor subsets behave like *phantom types* [4]: the extra information carried by the signature is present at type-checking time but is not reflected in the runtime representation of values.

Although phantom types are not widely known across programming languages and are used primarily in Haskell, they have important properties for representing types that carry no runtime information; that is, they are purely erased types [4]. In our system, there is no dedicated elimination rule for pure phantom signatures, and Rule 7 only permits case analysis on signatures that still expose constructors.

A constructor C must adhere to certain restrictions in its definition. For example, the signature (i.e., a type $\{T \Delta^* :: \Phi\}$) cannot occur freely in C . This restriction prohibits constructor types such as $\text{succ} : \{\text{nat} :: |\text{succ} \mid 0\} \rightarrow \{\text{nat} :: |\text{succ} \mid 0\}$, which would lead to problematic self-reference, as `succ` would require itself in its own definition.

It is important to note that even $\text{succ} : \text{nat} \rightarrow \text{nat}$ constitutes an invalid definition in this system. This is because its predecessor argument is a phantom type; therefore, for C to satisfy the inductive criteria, a signature must occur freely only in a positive position in the Δ of `succ`.

167 **Remark (inductive well-formedness).**

168 The previous paragraph sketches the kind of restrictions that an implementation must enforce
 169 so that inductive definitions remain well-founded (e.g. positivity/strictness conditions and
 170 well-scoped use of signatures in recursive arguments). Our prototype enforces a conservative
 171 check in the spirit of standard positivity conditions; however, we do not attempt to present a
 172 complete account of inductive well-formedness for signatures in this paper.

173 We do not cover index matching (from dependent (co)pattern matching) rules and conver-
 174 sion details in this work, as they are beyond our current focus. We leave the implementation
 175 of these principles to the authors' discretion.

$$\begin{array}{c} \Gamma \vdash T \Delta^* : \text{Type} \quad \Gamma \vdash Q : \text{Type} \quad \Gamma \vdash M : \{T \Delta^* :: \Phi\} \quad \Phi_{\text{ok}} = \text{OK}(\Gamma, T, \Delta^*, \Phi) \\ \Phi_{\text{ok}} \subseteq \Psi \subseteq \Phi \quad \Psi = (D_1, \dots, D_m) \\ \text{RULE 7} \frac{\forall k, 1 \leq k \leq m, \Gamma \vdash D_k : \Delta_{D_k} \rightarrow T \Delta_{D_k}^* \quad \forall k, 1 \leq k \leq m, \Gamma \vdash N_k : \Delta_{D_k} \rightarrow Q}{\Gamma \vdash \text{case } M \text{ of } Q \{ D_k \Delta_{D_k} \Rightarrow N_k \Delta_{D_k} \}_{k=1}^m : Q} \end{array}$$

176 Rule 7 requests branches only for constructors in a branch list Ψ that covers the obviously
 177 compatible constructors: $\Phi_{\text{ok}} \subseteq \Psi \subseteq \Phi$. In practice one chooses $\Psi = \Phi_{\text{ok}}$, which removes
 178 unreachable branches while keeping the rule conservative. There is yet a fancy way of dealing
 179 with signatures that contains trivial subsets, we could normalize them to their pruned version
 180 Φ_{ok} . We model this as an optional normalization step $\rightarrow_\phi$ on signature types.

181 Any conservative AGAINST definition that is sound for pruning is admissible: if it rejects
 182 a constructor at some index position, then Rule 3's index equality cannot hold for that
 183 constructor at those indices.

$$\text{NORM}_\phi \frac{\Phi_{\text{ok}} = \text{OK}(\Gamma, T, \Delta^*, \Phi)}{\{T \Delta^* :: \Phi\} \rightarrow_\phi \{T \Delta^* :: \Phi_{\text{ok}}\}}$$

184 As a further (optional) convenience, one may treat ϕ -normalization as part of defini-
 185 tional equality. We write $\Gamma \vdash A \equiv_{\beta\eta\phi} B$ for conversion that closes $\beta\eta$ -conversion with
 186 ϕ -simplification on signatures. This can be read operationally as: normalize both sides by
 187 $\beta\eta$ and repeatedly apply NORM_ϕ at signature nodes, then compare up to α -equivalence. We
 188 do not develop the meta-theory of $\equiv_{\beta\eta\phi}$ in this paper; although still simple to adhere to. In
 189 fact, many rules will be just simplified. It is presented as a design option that matches the
 190 implementation strategy of keeping signatures pruned. Although there is a cost of augmenting
 191 the complexity of the formalism, since you need to carry a new conversion relation.

$$\text{EQ}_\phi \frac{\Phi_{\text{ok}} = \text{OK}(\Gamma, T, \Delta^*, \Phi)}{\Gamma \vdash \{T \Delta^* :: \Phi\} \equiv_{\beta\eta\phi} \{T \Delta^* :: \Phi_{\text{ok}}\}}$$

$$\text{CONVERSION}_\phi \frac{\Gamma \vdash t : A \quad \Gamma \vdash B : s \quad \Gamma \vdash A \equiv_{\beta\eta\phi} B}{\Gamma \vdash t : B}$$

192 $\equiv_{\beta\eta\phi}$ is the smallest congruence containing $\beta\eta$ -conversion modulo Eq_ϕ . We state some
 193 properties of the rules 1-7 the system above:

194 ► **Theorem 1** (Head-constructor refinement). *Let T be an inductive family with constructor*
 195 *set C , and let $\Phi \subseteq C$ be a signature at indices Δ^* . Assume*

$$196 \quad \Gamma \vdash \{T \Delta^* :: \Phi\} : \text{Type} \quad \text{and} \quad \Phi = \Phi_{\text{ok}}.$$

197 *Define $D : T \Delta^* \rightarrow \text{Type}$ by inspecting only the head constructor of the canonical form: for*
 198 *each constructor application $C' \vec{u}$,*

$$199 \quad D(C' \vec{u}) \equiv \begin{cases} \top & \text{if } C' \in \Phi, \\ \perp & \text{if } C' \notin \Phi. \end{cases}$$

200 *Then there is a definable map*

$$201 \quad f : \{T \Delta^* :: \Phi\} \rightarrow \Sigma(x : T \Delta^*), D(x).$$

202 *Moreover, in any host theory that validates refinement introduction (from $x : T \Delta^*$ and $D(x)$*
 203 *to $x : \{T \Delta^* :: \Phi\}$), this map is part of an isomorphism $\{T \Delta^* :: \Phi\} \simeq \Sigma(x : T \Delta^*), D(x)$.*

204 **Proof.** Define $f(x) := (x, \star)$. To justify $\star : D(x)$, use the canonical-forms property for
 205 signature types: if $x : \{T \Delta^* :: \Phi\}$ is a value, its canonical form is $C \vec{u}$ with $C \in \Phi$, hence
 206 $D(C \vec{u}) \equiv \top$.

207 For the converse direction, assume refinement introduction: from $(y, d) : \Sigma(x : T \Delta^*), D(x)$
 208 we obtain $y : \{T \Delta^* :: \Phi\}$ and define $g(y, d) := y$. Then f and g are mutually inverse. ◀

$$\begin{array}{c} \xi\text{-APP}_1 \frac{L \longrightarrow L'}{L \cdot M \longrightarrow L' \cdot M} \qquad \xi\text{-APP}_2 \frac{M \longrightarrow M'}{V \cdot M \longrightarrow V \cdot M'} \\[10pt] \xi\text{-}\beta \frac{}{(\lambda x \Rightarrow N) \cdot V \longrightarrow N[x := V]} \end{array}$$

$$\xi\text{-case} \frac{v \longrightarrow v'}{\text{case } v \text{ of } Q \{C_i \Delta_i \Rightarrow N_i \Delta_i, \dots\} \longrightarrow \text{case } v' \text{ of } Q \{C_i \Delta_i \Rightarrow N_i \Delta_i, \dots\}}$$

$$\xi\text{-case}' \frac{}{\text{case } (C_i \Delta') \text{ of } Q \{C_i \Delta_i \Rightarrow N_i \Delta_i, \dots\} \longrightarrow N_i \Delta'}$$

209 Metavariables M, N, S range over terms; V, W range over values (lambda abstractions
 210 or constructor applications in WHNF). In the rule $\xi\text{-case}$ above, the symbols v, v' denote
 211 arbitrary terms (not values).

212 ► **Theorem 2** (Progress). *If $\vdash M : \{T \Delta^* :: \Phi\}$, then either M is a value or there exists M'*
 213 *such that $M \longrightarrow M'$.*

214 **Proof.** We reason by structural induction on M and case analysis on its form.

215 ■ **Application** $M = M_1 \cdot M_2$. If either M_1 or M_2 is not a value, the induction hypothesis
 216 yields M'_i with $M_i \longrightarrow M'_i$, and we use the corresponding congruence rule $\xi\text{-app}_i$ to obtain
 217 a step for M . Otherwise both M_1 and M_2 are values. If M_1 is a lambda abstraction
 218 $\lambda x. N$, we can perform a β -reduction step. If M_1 is a constructor, then M is already a
 219 value (an applied constructor) and the conclusion holds.

220 ■ *Case expression* $M = \text{case } v \text{ of } Q \{C_i \Delta_i \Rightarrow N_i \Delta_i, \dots\}$. If v is not a value, the induction
 221 hypothesis gives v' with $v \longrightarrow v'$, and we use rule ξ -case. If v is a value, typing gives
 222 $v : \{T \Delta^* :: \Phi\}$, so by canonical forms $v = C \vec{u}$ with $C \in \Phi$ and matching result indices.
 223 Therefore C is not rejected by AGAINST, hence $C \in \Phi_{\text{ok}}$. Rule 7 requires a branch list Ψ
 224 such that $\Phi_{\text{ok}} \subseteq \Psi$, so the branch for C is present and we apply ξ -case'.
 225 ■ *Other forms*. For variables, constructors, and abstractions the canonical forms analysis
 226 from the typing rules shows that well-typed closed terms of signature type are either
 227 values or reduce by one of the cases above.
 228 In all cases, a closed term of type $\{T \Delta^* :: \Phi\}$ is either a value or can take a reduction
 229 step. ◀

230 ► **Theorem 3 (Preservation).** *If $\Gamma \vdash M : R$ and $M \longrightarrow M'$, then $\Gamma \vdash M' : R$.*

231 **Proof.** We proceed by induction on the evaluation derivation, considering the last reduction
 232 rule used.

233 ■ β -reduction. If $M = (\lambda x.N).V$ and $M' = N[x := V]$, typing gives $\Gamma \vdash \lambda x.N : (x : A) \rightarrow B$
 234 and $\Gamma \vdash V : A$. By the substitution lemma we obtain $\Gamma \vdash N[x := V] : B[x := V]$, which
 235 is the same type as M .
 236 ■ ξ -app₁ and ξ -app₂. In these cases we reduce a proper subterm of M . The induction
 237 hypothesis states that the reduced subterm preserves its type; reapplying the application
 238 typing rule reconstructs a typing derivation for M' with the same type R .
 239 ■ ξ -case. Here $M = \text{case } v \text{ of } Q \{\dots\}$ and $M' = \text{case } v' \text{ of } Q \{\dots\}$ with $v \longrightarrow v'$. By the
 240 induction hypothesis $\Gamma \vdash v' : \{T \Delta^* :: \Phi\}$, and Rule 7 then re-establishes $\Gamma \vdash M' : Q = R$.
 241 ■ ξ -case'. In this case $M = \text{case } (C_i \Delta') \text{ of } Q \{\dots\}$ reduces to $M' = N_i \Delta'$. Rule 7 gives
 242 $\Phi_{\text{ok}} \subseteq \Psi \subseteq \Phi$ for the branch list and typing premises $\Gamma \vdash N_k : \Delta_{D_k} \rightarrow Q$ for every $D_k \in \Psi$.
 243 By canonical forms on the scrutinee, the head constructor satisfies $C_i \in \Phi_{\text{ok}}$, hence $C_i \in \Psi$
 244 and the corresponding branch exists. Instantiating with the actual arguments Δ' yields
 245 $\Gamma \vdash N_i \Delta' : Q$, so M' has the same type as M .

246 All other reduction rules follow the same pattern: either a proper subterm reduces and the
 247 induction hypothesis applies, or a local computation such as β -reduction is justified by the
 248 typing rules. In each case the result has the same type R as the original term. ◀

249 3 Case Examples

250 We introduce a minimal language and its implementation to demonstrate the subtyping
 251 system. The abstract syntax of terms is sketched in Figure 3; in the examples below we use
 252 a lightweight ML-style surface syntax.

$$\begin{aligned} \text{Term} ::= & \text{Var}(x) \mid \text{App}(f, a) \mid \text{Lam}(x, e) \mid \text{Pi}(x, A, B) \\ & \mid \text{Constr}(T, \vec{e}) \mid \text{Match}(e, T, p \rightarrow e') \mid \text{Notation}(e, T) \end{aligned}$$

■ **Figure 3** Term syntax of the example language

253 3.0.0.1 Mini-language overview.

254 We built a proof assistant in Haskell and implemented the type system described above,
 255 along with other features such as unification.

Programs consist of a sequence of *static declarations* followed by regular statements. Static declarations register constructors in $\mathcal{C}_{\text{all}}$ (Rule 1), whereas dynamic statements bind names to terms typed by the rules from Section 2. Terms use the usual λ -abstractions, function types, applications, and pattern matches; the only novelty is that constructor signatures $\{T :: \Phi\}$ appear explicitly in types. These informal descriptions suffice for the examples below; the full concrete syntax is implemented in the prototype but omitted here.

We exploit the capability to operate with dynamically defined constructor subsets without requiring trivial proof obligations. Since datatype signatures are first-class inhabitants in our system, we can instantiate varying constructor sets through definition aliasing. The following declarations register constructors for lists and expose two different signatures:

```

267 1 Static list : * > *.
268 2 Static empty : (A : *) -> (list A).
269 3 Static new : (A : *) -> A > {(list A) :: |new |empty|} > (list A).
270 4 List |A :: * > * => {(list A) :: |new |empty|.
271 5 NonEmpty |A :: * > * => {(list A) :: |new|.
272

```

■ **Listing 1** Definition of empty and non-empty lists using first-class signatures

Rule 3 ensures that each constructor inhabits the declared signature, while Rule 4 provides $\text{NonEmpty } A \sqsubseteq \text{List } A$ because the latter merely adds the `empty` constructor. This formulation enables the definition of functions that operate on both general lists and non-empty lists:

```

276
277 1 length |A ls :: (A : *) -> (List A) > Nat =>
278 2 [ls of Nat
279 3 |(empty _) => 0
280 4 |(new A head tail) => (+1 (length A tail))
281 5 ].
282

```

■ **Listing 2** Length function compatible with both list variants

Alternatively, we can define functions that exclusively accept non-empty lists, directly mirroring the running example:

```

285
286 1 last |A ls :: (A : *) -> (NonEmpty A) > A =>
287 2 [ls of A
288 3 |(new A head tail) => [tail of A
289 4 |(empty _) => head
290 5 |(new A head2 tail2) => (last A (new A head2 tail2))
291 6 ]
292 7 ].
293 8 insertsort |xs v :: (List Nat) > Nat > (NonEmpty Nat) =>
294 9 [xs of (NonEmpty Nat)
295 10 |(empty _) => (new Nat v (empty Nat))
296 11 |(new _ head tail) => [(gte head v) of (NonEmpty Nat)
297 12 |false => (new Nat head (insertsort tail v))
298 13 |true => (new Nat v (new Nat head tail))
299 14 ]
300 15 ].
301 16 def list_inserted_has_a_last_element |xs v :: (List Nat) > Nat > Nat =>
302 17 (last Nat (insertsort xs v))
303

```

■ **Listing 3** Subtyping application to eliminate trivial proof obligations

Rules 2 and 7 explain why `last` needs only one clause: the type of `ls` is $\{(List\ A) :: |new|\}$, so no `empty` branch is requested. The helper `list_inserted_has_a_last_element`

XX:10 First-Class Constructor Subsets for Pattern Matching on Indexed Families

306 demonstrates that the sorted list inherits the non-empty signature with no auxiliary proofs—
 307 exactly the two obligations eliminated relative to Figure 1.

308 One might assume the following representation is correct, as previously discussed, yet
 309 Rule 2 warns against using phantom predecessors:

```
310
311 1 Static nat : *.
312 2 Static 0 : nat.
313 3 Static +1 : nat > nat.
314 4 Nat :: {nat :: |0 |+1}.
```

■ **Listing 4** Attempt with phantom predecessor

316 Here the predecessor of `+1` would live in $\{nat :: |0| + 1\}$ but the constructor type mentions
 317 only the phantom `nat`. The checker’s algorithmic Rule 5 rejects this definition, forcing us to
 318 use the recursive variant below:

```
319
320 1 Static nat : *.
321 2 Static 0 : nat.
322 3 Static +1 : Nat > nat.
323 4 Nat :: {nat :: |0 |+1}.
```

■ **Listing 5** Recursive definition accepted by the algorithmic Rule 5 check

325 Because the predecessor now mentions the signature `Nat`, Rule 3 can derive the constructor
 326 typing judgment directly. The pruning function used by Rule 7 then removes obviously
 327 incompatible branches when matching on such signatures, avoiding unnecessary proof obliga-
 328 tions [9].

329 4 Implementation

330 Our implementation now has three components. The first component is the minimal language
 331 implemented above, which supports inductive datatypes in a superset of the $\lambda\Pi$ calculus
 332 modulo (Figure 4).

$$\begin{array}{c}
 \text{SORT} \frac{}{\Gamma; \Delta \vdash \text{Type} : \text{Kind}} \quad \text{VARIABLE} \frac{(x : A) \in \Delta}{\Gamma; \Delta \vdash x : A} \quad \text{CONSTANT} \frac{(c : A) \in \Gamma}{\Gamma; \Delta \vdash c : A} \\
 \\
 \text{APPLICATION} \frac{\Gamma; \Delta \vdash t : \Pi x : A. B \quad \Gamma; \Delta \vdash u : A}{\Gamma; \Delta \vdash tu : B[x/u]} \\
 \\
 \text{ABSTRACTION} \frac{\Gamma; \Delta \vdash A : \text{Type} \quad \Gamma; \Delta(x : A) \vdash t : B \quad B \neq \text{Kind}}{\Gamma; \Delta \vdash \lambda x : A. t : \Pi x : A. B} \\
 \\
 \text{PRODUCT} \frac{\Gamma; \Delta \vdash A : \text{Type} \quad \Gamma; \Delta(x : A) \vdash B : s}{\Gamma; \Delta \vdash \Pi x : A. B : s} \\
 \\
 \text{CONVERSION} \frac{\Gamma; \Delta \vdash t : A \quad \Gamma; \Delta \vdash B : s \quad A \equiv_{\beta\Gamma} B}{\Gamma; \Delta \vdash t : B}
 \end{array}$$

■ **Figure 4** Typing rules for terms in the $\lambda\Pi$ -calculus Modulo (Figure 3 in [12]).

The backend encoding in `Lambdapi` (`encode.lp`, about 600 lines) defines signatures, constructor-list well-formedness, subtyping checks, and signature-directed elimination on top of the $\lambda\Pi$ calculus modulo rewriting [12]; it is a shallow encoding of the rules from Section 2. The frontend transpiler parses a restricted surface syntax and lowers it to backend terms. In fact, it transpiles only a fragment of the minimal language proposed, but this is enough to demonstrate the simplicity and expressiveness of the system. The lowering mirrors Rules 1–7 from Section 2. Signature annotations become `signature_term`. Constructor introductions become `signature_intro`. Signature-only matches become `signature_case`. Appendix A shows representative input and generated output.

5 Conclusion

In this research, we introduced a compact yet powerful subtype system that demonstrates remarkable versatility and compatibility with various type theories. Our system effectively addresses the challenge of reusing and subtyping constructors in indexed datatypes through the novel introduction of first-class datatype signatures. A particularly notable feature of our approach is its ability to eliminate trivial proof obligations even when an arbitrary set of constructors is involved, significantly reducing the proof burden in practical applications. We have illustrated the system’s flexibility through several practical examples drawn from real-world programming scenarios. Furthermore, we have provided a concise implementation that maintains simplicity without sacrificing expressiveness, making our approach accessible for integration into existing proof assistants and type systems.

6 Future Work

There are several directions in which we would like to extend this work.

- *Mechanized meta-theory.* We plan to formalize the typing and subtyping rules, together with subject reduction and progress, in a proof assistant such as Coq or Agda, and connect the proofs to our prototype implementation.
- *Larger case studies.* Our current examples are small; applying constructor subset signatures to larger developments in existing proof assistants (e.g. libraries of lists and vectors) would help validate their practical benefits.
- *Richer signatures.* Finally, we intend to explore simple extensions of signatures with basic set operations on constructor sets (such as unions) while keeping the meta-theory and implementation lightweight.

A Surface-to-Lambdapi Transpiler

The current artifact is organized around a translation pipeline: users write a compact surface fragment, and `siglp-transpiler` lowers it to backend terms consumed by `subtyping.encode`. This keeps examples short while preserving explicit generated code.

The supported surface fragment translates `LambdaPi Inductive` declarations. We compile the `Def` declarations, signature shorthand (`{T :: c_1 | ... | c_n}`), and signature-only `match` expressions. For each inductive declaration, the transpiler generates helper symbols for target, index shape, telescope, and constructor list. It rewrites term-level signatures to `signature_term`. It rewrites constructor applications to `signature_intro`. It rewrites signature matches to `signature_case`.

XX:12 First-Class Constructor Subsets for Pattern Matching on Indexed Families

```
1 Static nat : *.
2 Static 0 : nat.
3 Static +1 : Nat -> nat.
4 Nat :: {nat :: |0 |+1}.
5
6 Static vec : * -> nat -> *.
7 Static empty : (s : *) -> (vec s 0).
8 Static cons : (s : *) -> (n : nat) -> (vec s n) -> (lift s) -> (vec s
  (+1 n)).
9
10 NonEmptyVecNat |v :: nat -> * => {(vec nat (+1 v)) :: |cons}.
11
12 head_nonempty |v m :: nat -> (NonEmptyVecNat v) -> Nat =>
13 [m of Nat
14  |(cons k) => (+1 0)
15 ].
```

■ **Figure 5** Mini-language rendering of the non-empty vector example(*The syntax was slightly modified to match the implementation of the minimal language in Haskell*)

374 Compiling with `cabal run siglp-transpiler` (input: `examples/sample.siglp`, out-
375 put: `examples/sample.lp`) yields LambdaPi code such as Figure 6.

```
require open subtyping.encode;

symbol vec__sig_target : Set;
rule vec__sig_target ⇔ vecLabel;
symbol vec__sig_indexes : indexes_type;
rule vec__sig_indexes ⇔
  index_poly_succ (index_dependent_lift_succ natLabel (index_null));
symbol vec__sig_delta : delta_tel;
rule vec__sig_delta ⇔
  mk_delta_tel vec__sig_indexes vec__sig_pattern_any;

symbol head_nonempty : Π (v : nat),
  (signature_term vec__sig_target
    (mk_delta_tel vec__sig_indexes (...))
    (constructor_list_cons consLabel (constructor_list_nil))) → nat;

rule head_nonempty ⇔ λ (v : nat) (m : ...),
  (signature_case vec__sig_target (mk_delta_tel vec__sig_indexes (...))
    (constructor_list_cons consLabel (constructor_list_nil))
    natLabel m
    (case_branches_cons consLabel (constructor_list_nil) natLabel
      (λ (k : constructor_type consLabel), (+1 _0))
      (case_branches_nil natLabel)));
```

■ **Figure 6** Generated LambdaPi excerpt after lowering signatures and matches

376 The complete generated files are omitted for space. Full examples are available in
377 `subindex/subtyping/transpiler/examples/`, and the target encoding used by generated
378 terms is `subindex/subtyping/encode.lp`.

379 Backend shallow encoding (encode.lp).

380 The backend is organized as a shallow encoding of Rules 1–7 into LambdaPi symbols and
381 rewrite rules. Formation and well-formedness are captured by `signature_wfb` and `signature`.
382 Subtyping uses boolean checkers (`signature_subtypeb`, `signature_to_base_subtypeb`)

383 wrapped by `is_true` when a proposition is required. Elimination is represented directly by
 384 `signature_term`, `case_branches`, and `signature_case`. Finally, a coercion hook (`coerce_rule`)
 385 inserts signature widening automatically when constructor-list inclusion is validated.

```

symbol signature_wfb : Set → delta_tel → constructor_list → Bool;
inductive signature : Set → delta_tel → constructor_list → TYPE :=
| signature_nil : Π (T : Set) (g : delta_tel), signature T g constructor_list_nil
| signature_cons :
  Π (T : Set) (g : delta_tel) (c : labelConstructor) (cs : constructor_list),
  π (is_true (against_delta_b T g (constructor c))) →
  signature T g cs →
  signature T g (constructor_list_cons c cs);

symbol signature_subtypeb :
  Set → delta_tel → constructor_list →
  Set → delta_tel → constructor_list → Bool;

inductive signature_term : Set → delta_tel → constructor_list → TYPE :=
| signature_intro :
  Π (T : Set) (g : delta_tel) (phi : constructor_list) (c : labelConstructor),
  π (is_true (constructor_in_signature_b T g phi c)) →
  constructor_type c →
  signature_term T g phi;

symbol signature_case :
  Π (T : Set) (g : delta_tel) (phi : constructor_list) (Q : Set),
  signature_term T g phi →
  case_branches phi Q →
  lift Q;

coerce_rule coerce_signature
  (signature $T $g $phi1)
  (signature $T $g $phi2)
  $x ↦ coerce_signature $T $g $phi1 $phi2 $x;

```

■ Figure 7 Shallow backend encoding excerpt from `encode.lp`

386 A.1 IA aid

387 Although the transpiler was made with IA aid, the mini language was before the popularization
 388 of the tool, and `lambdapi` encoding was done manually.

389 References

- 390 1 D. Aspinall and A. Compagnoni. Subtyping dependent types. In *Proceedings 11th Annual*
 391 *IEEE Symposium on Logic in Computer Science*, pages 86–97, 1996. doi:10.1109/LICS.1996.
 392 561307.
- 393 2 Gilles Barthe. Order-sorted inductive types. *Information and Computation*, 149(1):42–76,
 394 1999. URL: <https://www.sciencedirect.com/science/article/pii/S0890540198927511>,
 395 doi:<https://doi.org/10.1006/inco.1998.2751>.
- 396 3 Gilles Barthe and Maria João Frade. Constructor subtyping. In *Proceedings of the 8th*
 397 *European Symposium on Programming Languages and Systems*, ESOP '99, page 109–127,
 398 Berlin, Heidelberg, 1999. Springer-Verlag.
- 399 4 Matthew Fluet and Riccardo Pucella. Practical datatype specializations with phantom types
 400 and recursion schemes, 2005. arXiv:cs/0510074.
- 401 5 Maria João Frade. Type-based termination of recursive definitions and constructor subtyping in
 402 typed lambda calculi. 2003. URL: <https://api.semanticscholar.org/CorpusID:115763806>.
- 403 6 Christopher Jenkins, Colin McDonald, and Aaron Stump. Elaborating inductive definitions
 404 and course-of-values induction in cedille, 2019. arXiv:1903.08233.

- 405 **7** Andrew Marmaduke, Christopher Jenkins, and Aaron Stump. Zero-cost constructor subtyping.
406 In Olaf Chitil, editor, *IFL 2020: 32nd Symposium on Implementation and Application of*
407 *Functional Languages, Virtual Event / Canterbury, UK, September 2-4, 2020*, pages 93–103.
408 ACM, 2020. doi:10.1145/3462172.3462194.
- 409 **8** Rodrigo Marques, Mário Florido, and Pedro Vasconcelos. Towards algebraic subtyping for
410 extensible records, 2024. URL: <https://arxiv.org/abs/2407.06747>, arXiv:2407.06747.
- 411 **9** Per Martin-Löf. Intuitionistic type theory by per martin-löf. notes by giovanni sambin of a
412 series of lectures given in padova, june 1980. 2021.
- 413 **10** Johan Nordlander. Polymorphic subtyping in o’haskell. *Science of Computer Program-*
414 *ming*, 43(2-3):93–127, 2002. Validerad; 2002; 20070227 (ysko). doi:10.1016/S0167-6423(02)
415 00026-6.
- 416 **11** Ulf Norell. Towards a practical programming language based on dependent type theory. 2007.
417 URL: <https://api.semanticscholar.org/CorpusID:118357515>.
- 418 **12** Ronan Saillard. Typechecking in the lambda-pi-calculus modulo : Theory and practice.
419 (vérification de typage pour le lambda-pi-calcul modulo : théorie et pratique). 2015. URL:
420 <https://api.semanticscholar.org/CorpusID:2853829>.